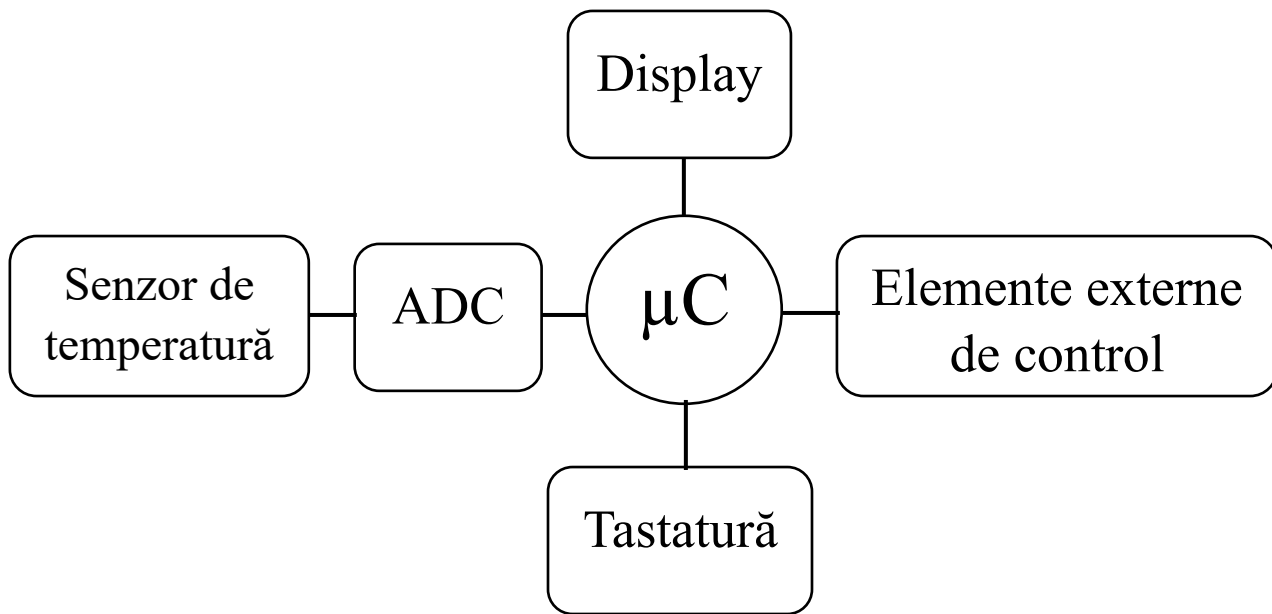


Proiect microcontrolere

Termostatul



Proiect realizat de: Simedroni Andra

1. Prezentare generală

Scopul proiectului este de a realiza un termosta de cameră, care folosește un microcontroler, un senzor de temperatură, o tastatură, elemente externe de control și un display.

2. Senzori de temperatură

În ceea ce privește senzorul de temperatură, acesta trebuie să funcționeze într-un interval cuprins între 5°C și 30°C , cu o precizie de 0.2%. Pentru găsirea senzorului potrivit, voi explica funcționarea următoarelor categorii de senzori:

2.1 Senzori rezistivi

Senzorii rezistivi (Eng. *RTD sensors* – “Resistance Temperature Detector”) sunt dispozitive de măsurare a temperaturii a căror rezistență electrică variază liniar și direct proporțional cu temperatura. Aceștia se bazează pe proprietatea materialelor conductoare de a-și modifica **rezistivitatea electrică (ρ)** sub influența temperaturii. Materialul utilizat pentru elementul sensibil trebuie să aibă o rezistivitate stabilă și bine definită, motiv pentru care **platina** este cel mai utilizat material. Aceasta prezintă o stabilitate excelentă pe termen lung, un interval larg de operare și o relație aproape liniară între rezistență și temperatură. În plus, coeficientul de temperatură al platinei este bine determinat, ceea ce permite o calibrare precisă.

Creșterea temperaturii determină o **creștere a rezistivității** metalului, ceea ce implică o **creștere a rezistenței**. Acest fenomen este cauzat de interacțiunile electronilor de conducție cu rețeaua cristalină a materialului, ceea ce duce la o creștere a rezistenței la trecerea curentului electric. Astfel, pe măsură ce temperatura crește, viteza medie de deplasare a electronilor este redusă din cauza dispersiei crescute, ducând la o **scădere a intensității curentului electric** pentru un potențial electric constant.

Avantaje: precizie ridicată, fiabilitate pe termen lung, interval de temperatură larg, stabilitate termică superioară.

Dezavantaje: cost ridicat, timp de răspuns lung, fragilitate în cazul senzorilor cu film subțire.

2.2 Termocupluri

Termocuplurile sunt senzori de temperatură compuși din două metale diferite unite la un capăt, formând o **joncțiune termoelectrică**. Când aceasta e expusă la o temperatură diferită față de cealaltă extremitate a circuitului, se generează o **tensiune electrică** proporțională cu diferența de temperatură. Această tensiune este utilizată pentru a determina temperatura măsurată.

Funcționarea termocuplurilor se bazează pe **efectul Seebeck**, un fenomen termoelectric conform căruia, atunci când două metale diferite sunt conectate și expuse la temperaturi diferite, se creează o **forță electromotoare (FEM)**, direct proporțională cu diferența de temperatură dintre cele două extremități. Datorită acestui effect, nu e necesară o sursă externă de alimentare.

Avantaje: gamă largă de temperaturi, timp de răspuns rapid, robustețe și durabilitate, cost redus, nu necesită sursă de alimentare.

Dezavantaje: precizie scăzută, drift termic (pot suferii degradare în timp), sensibilitate la interferențe electromagnetice, necesitatea unei temperaturi de referință.

2.3 Termistori

Termistorii sunt rezistori semiconductori, a căror rezistență variază în funcție de temperatură. Această proprietate se datorează materialului utilizat în construcția lor, de obicei **oxizi de metal**, cum ar fi oxidul de mangan, nichel sau cobalt. În comparație cu **detectoarele de temperatură cu rezistență (RTD)**, termistorii sunt **mai sensibili**, dar au o **dependență neliniară de temperatură**, ceea ce poate necesita calibrare suplimentară.

În general, termistorii sunt separați în două categorii, în funcție de **comportamentul termic**:

- **Cu coeficient negativ de temperatură (NTC):** rezistența scade pe măsură ce temperatura crește. Factorul β (coeficientul termic) e foarte important, fiind o constantă a materialului ceramic din care e realizat termistorul, care descrie rata de variație a rezistenței cu temperatura. Sunt utilizați frecvent în **măsurarea temperaturii**, circuite de compensare termică și protecție la supracurent.
- **Cu coeficient pozitiv de temperatură (PTC) :** rezistența crește atunci când temperatura crește. Sunt fabricați din **ceramică policristalină** și pot deveni **semiconductori** atunci când sunt **dopați**. Folosiți pentru **protecția la supracurent, limitarea curentului de pornire și declanșatoare termice**.

Avantaje: senzibilitate ridicată la schimbări de temperatură, dimensiuni reduse și răspuns rapid, cost redus și gamă largă de temperaturi de operare.

Dezavantaje: neliniaritate ridicată, sensibilitate la variații de tensiune și curent, toleranțe mari, care limitează precizia.

2.4 Senzori cu semiconductori

Senzorii cu semiconductori sunt bazați pe sensibilitatea cu temperatura a dispozitivelor cu siliciu. Se bazează pe variația tensiunii de deschidere a unei joncțiuni p-n cu temperatura:

$$V_D = \frac{K \cdot T}{q} \cdot \ln \frac{I_D}{I_S}$$

unde: K este constanta lui Boltzman, q este sarcina electronului, T temperatura absolută a joncțiunii, I_S curentul de saturație iar I_D este curentul direct prin joncțiune. Totuși, utilizarea acestei formule pentru variații largi nu e indicată, deoarece și curentul de saturație depinde de temperatură. Astfel, se pot utiliza **diode** sau **tranzistori** ca senzori de temperatură.

În cazul **diodelor**, se aplică un curent constant pe diodă pentru a obține o tensiune proporțională cu temperatura, se măsoară tensiunea directă a diodei, care scade cu aproximativ $2\text{mV}/^\circ\text{C}$ și se poate folosi un amplificator operațional pentru a converti această tensiune într-un semnal ușor de procesat.

În cazul **tranzistoarelor**, joncțiunea **bază-emitor** a unui tranzistor bipolar (**BJT**) funcționează similar unei diode și poate fi utilizată pentru măsurarea temperaturii: se polarizează joncțiunea bază-emitor cu un curent constant, se măsoară tensiunea dintre bază și emitor, care variază cu temperatura într-un mod predicibil și se poate utiliza un circuit diferențial cu 2 tranzistoare pentru îmbunătățirea preciziei, cum este în cazul senzorilor integrați din microprocesoare.

2.5 Senzori de temperatură cu infraroșu

Senzorii de temperatură cu infraroșu măsoară radiația termică emisă de un obiect și determină temperatura fără contact fizic. Aceștia funcționează pe baza legii radiației corpului negru a lui Planck și a legii lui Boltzmann, conform căreia orice obiect cu o temperatură peste zero absolut emite radiații infraroșii proporționale cu temperatura sa.

Un senzor IR de temperatură este alcătuit din: un **detector piroelectric** sau **termopil**, care convertește radiația infraroșie în semnal electric, un filtru optic, care permite doar radiația cu anumite lungimi de undă să ajungă la detector și un circuit de prelucrare a semnalului, care convertește semnalul analogic într-o valoare digitală corespunzătoare temperaturii.

Avantaje: timp de răspuns rapid, gamă largă de temperaturi măsurabile.

Dezavantaje: sensibilitate ridicată la variațiile curentului aplicat, necesită calibrare pentru a compensa variațiile din fabricație, pot introduce erori din cauza autoîncălzirii dacă nu sunt dimensionate corect.

2.6 Alegerea senzorului pentru termostat

Având în vedere datele prezentate mai sus, am ales să folosesc un **senzor semiconductor**, datorită ieșirii liniare, a preciziei și a stabilității pe termen lung, a costului redus și a ușurinței de integrare cu microcontrolerul 80C51 și a gamei largi de temperaturi pe care o acoperă.

Senzor semiconductor	Liniaritate	Precizie	Gamă de temperaturi	Tip de ieșire	Alimentare	Preț
LM35H	$\pm 10\text{-mV}/^{\circ}\text{C}$	$\pm 0.5\text{ }^{\circ}\text{C}$	-55 C +150 C	Tensiune	4V 30V	81.53 RON
LM235H	$\pm 10\text{-mV}/^{\circ}\text{C}$	$\pm 1\text{ }^{\circ}\text{C}$	-40 C +125 C	Tensiune	2.99V 5V	37.92 RON
LM60BIZ	$6.25\text{mV}/^{\circ}\text{C}$	$\pm 3\text{ }^{\circ}\text{C}$	-40 C +125 C	Tensiune	2.7V 10V	4.33 RON
926-LM35DT	$\pm 10\text{-mV}/^{\circ}\text{C}$	$\pm 0.5\text{ }^{\circ}\text{C}$	0 C 100 C	Tensiune	4V 30V	9.55 RON
926-LM35DZ	$\pm 10\text{-mV}/^{\circ}\text{C}$	$\pm 0.5\text{ }^{\circ}\text{C}$	0 C 100 C	Tensiune	4V 30V	5.99 RON

Din senzorii semiconductori de mai sus, consider că **926-LM35DZ** este cea mai potrivită variantă pentru realizarea termostatului, deoarece acesta conferă o ieșire liniară, cu o **sensibilitate** de $10\text{ mV}/^{\circ}\text{C}$, ceea ce înseamnă că la fiecare grad Celsius, semnalul va varia cu 10 mV . **Precizia** de 0.5°C asigură o **performanță** suficient de bună pentru aplicația dată. Există compatibilitate între **alimentarea** microcontrolerului 80C51 (alimentat la 5 V) și senzorul ales. Versiunea **through-hole** pe care am ales-o e ușor de manipulat și de conectat, ceea ce face proiectul ușor de realizat practic, în laborator. Senzorul este destul de **stabil în timp** și are o **performanță constantă**, iar **prețul** este mic.

3. Conectarea senzorului la ADC

3.1 ADC0808

ADC0808 este un **convertor analog-digital pe 8 biți**, cu 8 canale de intrare analogică. Poate opera cu o tensiune de referință de 5V (implicit). Astfel, domeniul de tensiuni este [0V – 5V].

Fiind un convertor pe 8 biți, ADC0808 poate prezenta $2^8 = 256$ de niveluri discrete (pași de cuantizare). **Dimensiunea pasului de cuantizare** reprezintă cea mai mică diferență de tensiune care poate fi detectată și convertită de ADC și se calculează astfel:

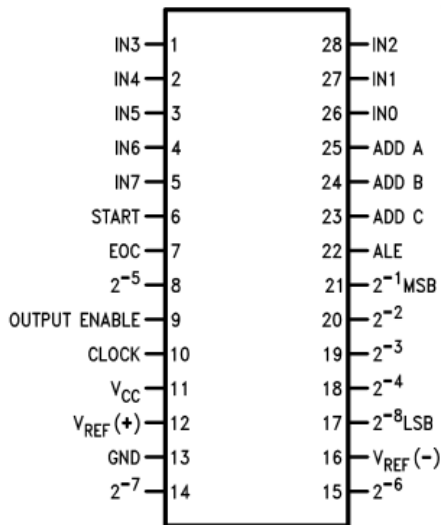
$$V_{LSB} = \frac{V_{FS}}{2^8} = \frac{5V}{256} \cong 20mV$$

Ca efect apare **eroarea de cuantizare**, datorată faptului că un semnal analogic are un număr finit de niveluri. Dacă nu există o valoare exactă pe care ADC-ul o poate reprezenta cel mai bine, va aproxima cel mai apropiat nivel discret, introducând eroarea precizată mai sus.

Intrarea care urmează să fie convertită în formă digitală poate fi selectată utilizând 3 linii de adresă.

ADC0808 necesită un ceas extern pentru a funcționa și necesită anumite semnale de control specifice pentru operațiunile sale, cum ar fi pornirea conversiei și aducerea datelor la pinii de ieșire.

Descrierea pinilor:

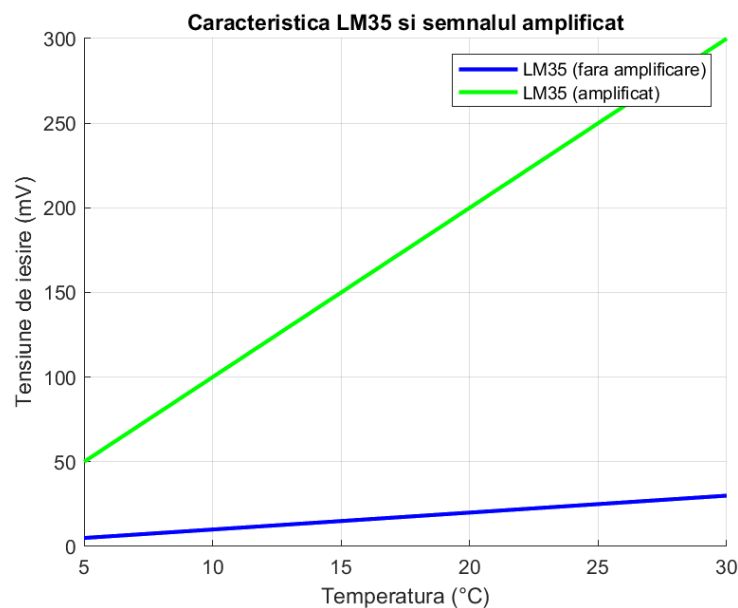


- **V_{CC}**: tensiunea de alimentare(+5V)
- **GND**: împământarea
- **V_{REF}(+)/V_{REF}(-)**: pinii de referință pentru tensiune
- **In0 - In7**: Canale de intrare analogică
- **A, B, C**: linii de adresă pentru selectarea canalului de intrare
- **ALE**: folosit pentru a încărca adresa
- **START**: semnal pentru inițierea conversiei
- **EOC**: indică finalizarea conversiei
- **OE**: output enable – permite ieșirea datelor convertite
- **D0 – D7**: pini de ieșire pentru datele digitale

Pentru a **utiliza** ADC0808:

- se selectează **canalul de intrare analogică** utilizând liniile de adresă (**A, B, C**)
- se trimite un impuls pe pinul **ALE** pentru a încărca adresa
- se pornește conversia la aplicarea unui impuls pe pinul **START**
- conversia se finalizează când pinul **EOC** devine **LOW**
- pentru a permite **ieșirea datelor** se activează pinul **OE** și se citește valoarea digitală de pe pinii **D0 – D7**

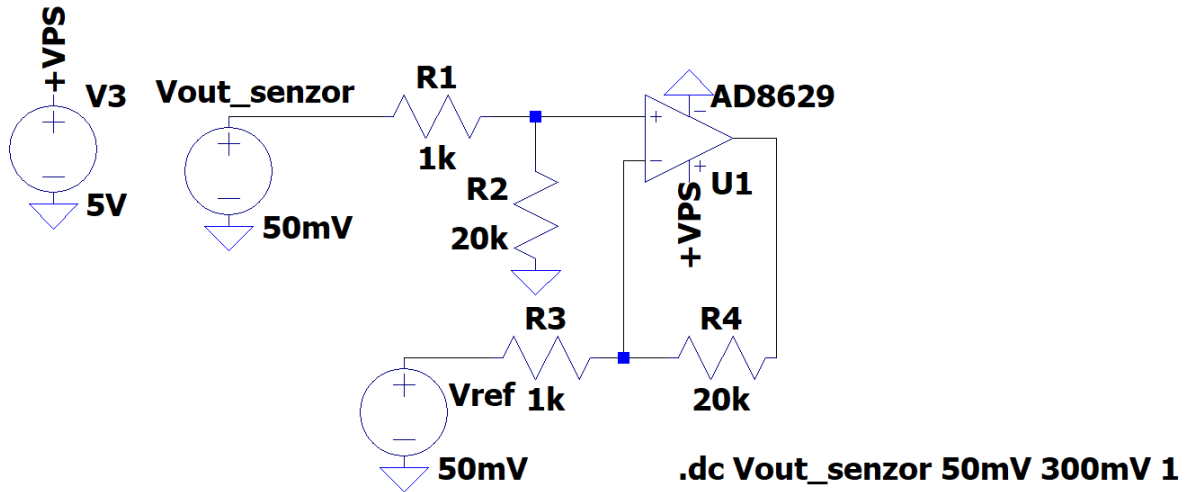
În cazul termostatlui, precizia oferită de senzorul **LM35** în jurul temperaturii de 25°C este de $2\text{mV}/^{\circ}\text{C}$, ceea ce înseamnă că pasul de cuantizare al ADC-ului este mult mai mare față de pasul senzorului de temperatură, iar acest lucru poate cauza **erori de precizie**. Pentru a rezolva această problemă, va trebui ca semnalul analogic de la ieșirea senzorului de temperatură să fie **amplificat** după cum urmează:



Această caracteristică arată valoarea semnalului de la LM35 cu și fără amplificarea dată de circuitul de mai jos.

3.2 Circuit de amplificare pentru îmbunătățirea preciziei ADC0808

Pentru a amplifica semnalul de la ieșirea senzorului LM35, se propune următorul circuit:



Pentru dimensionarea acestui amplificator diferențial, am considerat tensiunea de intrare (adică ieșirea senzorului) pentru cazurile 0V și 30V:

$$V_{out} = \frac{R4}{R3} \cdot (V_{out_{senzor}} - V_{ref})$$

$$\text{Caz 1: } V_{out} = 0V$$

$$\rightarrow 0 = \frac{R4}{R3} \cdot (50mV - V_{ref})$$

$$\rightarrow V_{ref} = 50mV$$

$$\text{Caz 2: } V_{out} = 5V$$

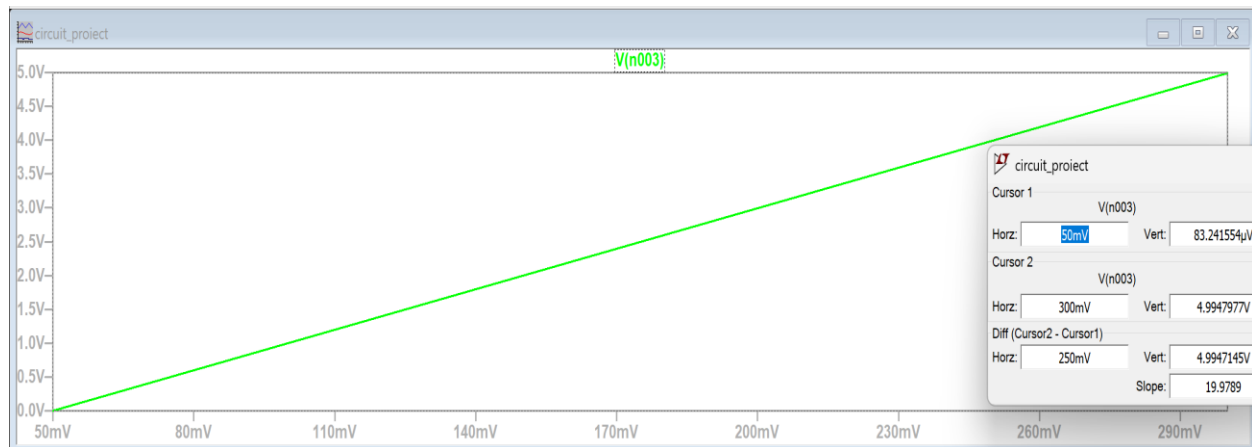
$$\rightarrow 5V = \frac{R4}{R3} \cdot (300mV - 50mV)$$

$$\rightarrow \frac{R4}{R3} = \frac{5V}{250mV}$$

$$\rightarrow \frac{R4}{R3} = 20$$

Astfel, am ales valoarea $R3 = 1k\Omega$ și $R4 = 20k\Omega$ și se știe că o condiție ca acest circuit să fie amplificator diferențial este ca $R3 = R1$ și $R2 = R4$. În urma unei analize DC sweep în LTspice, am obținut următorul rezultat, care confirmă faptul că circuitul în configurație de amplificator

diferențial a modificat domeniul de variație al tensiunii de ieșire de la senzorul de temperatură convenabil pentru a putea fi citit de către convertorul ADC0808:



Am ales să folosesc amplificatorul AD8629 deoarece:

- este single supply
- are un offset de tensiune foarte mic, de $1\mu\text{V}$, ceea ce înseamnă că nu introduce erori în măsurători
- are un PSRR foarte mare, $\sim 120\text{ dB}$, nefiind afectat de variațiile sursei de alimentare
- SR-ul de $1.7\text{ V}/\mu\text{s}$ este suficient de bun pentru un termostat, deoarece semnalul analogic al senzorului este linear, deci nu are variații bruște care să fie greu urmărite de OA, iar ADC-ul citește valori cu schimbări lente

Se poate observa și din analiza LT spice că există un offset foarte mic, aproape insesizabil, de $83,24\mu\text{V}$, adică circuitul are un offset foarte mic care nu va afecta măsurătorile ADC-ului.

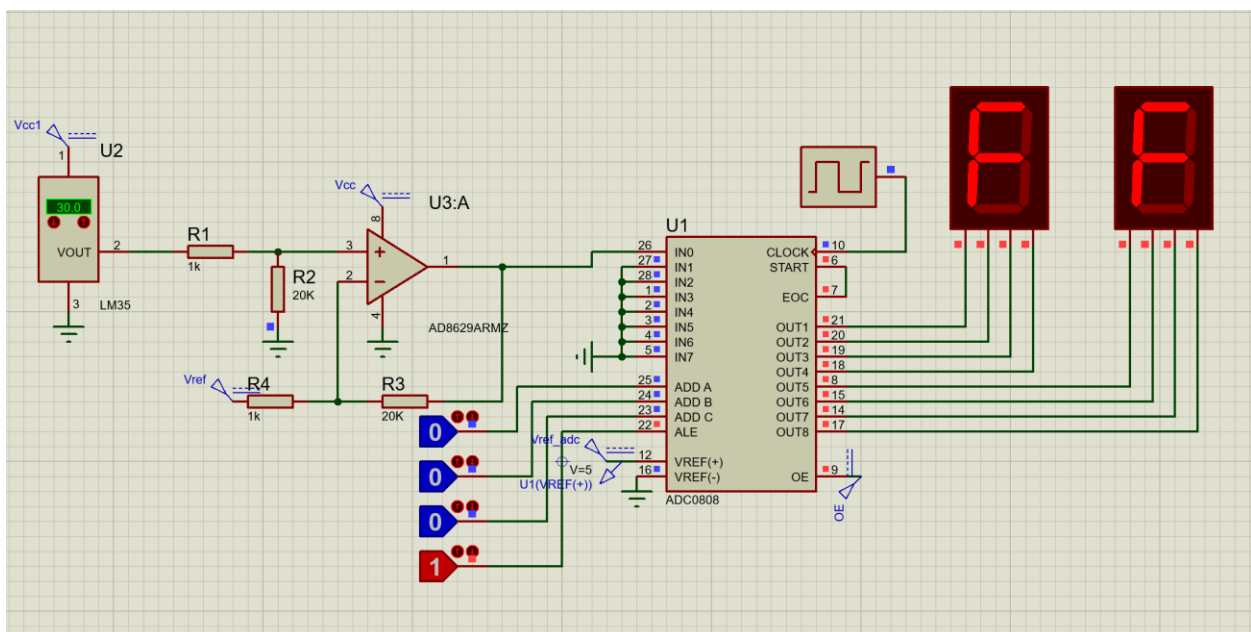
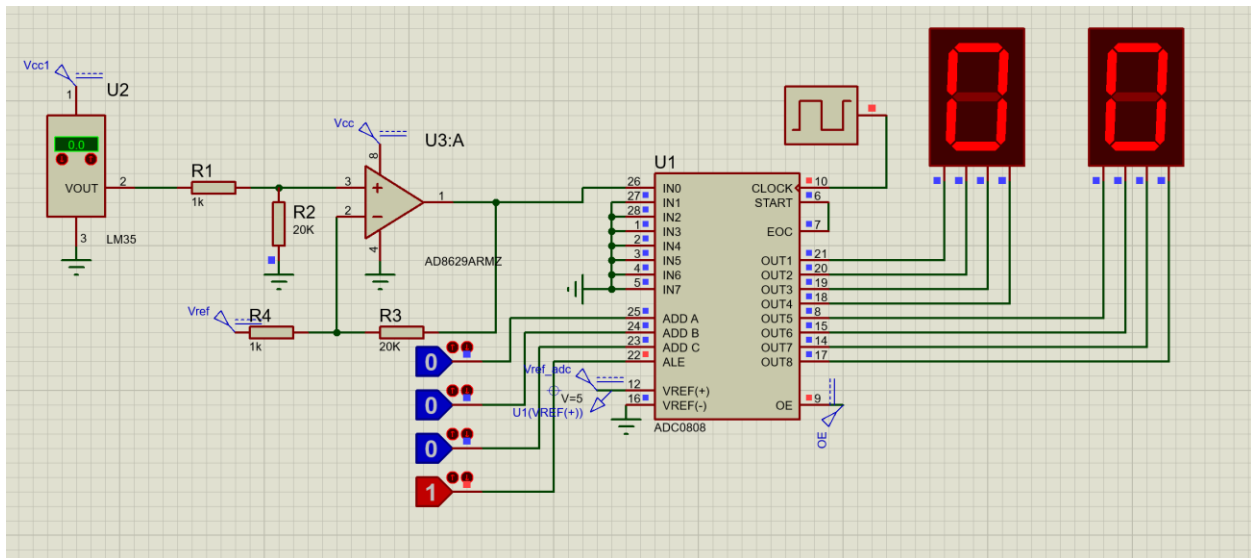
3.3 Implementare în Proteus

Pentru implementarea în Proteus, am folosit schema electrică prezentată mai sus, dar am conectat după cum urmează:

- In0 – Outputul de la OA
- ADD A, ADD B și ADD C sunt conectate la LOGICSTATE și formează 000, pentru a indica conectarea senzorului și a circuitului diferențial la portul 0
- ALE – LOGICSTATE pe 1, pentru a încărca C, B, A

- EOC și START – conectate împreună pentru a începe conversia imediat ce ADC-ul e pornit
- Outputurile la 7SEG
- CLOCK – conectat la un clock generator de 640KHz (valoarea tipică de care are nevoie ADC0808 la clock-ul extern pentru a funcționa)
- VREF(+) la 5V
- VREF(-) la GND
- IN1-7 la GND

La aplicarea unei temperaturi de 0°C am obținut 00 la ieșirea 7Segment-ului, iar la aplicarea unei temperaturi de 30°C am obținut FF.



4. Microcontrolerul 8051

4.1 Familia 8051

Stă la baza standardului industrial pentru microcontrolerele performante pe 8 biți. Intel a introdus această familie sub denumirea MCS-51, nume derivat inițial de la tehnologia NMOS utilizată în fabricarea microcontrolerului, fiind completată apoi cu tehnologia CMOS.

Microcontroler	RAM	ROM integrat	Nr. de timere	Înteruperi	Porturi seriale	Nr. porturi I/O
8051	128 bytes	4KB	2	6	1	32
8052	256 bytes	8KB	3	8	1	32
8031	128 bytes	0KB	2	6	1	32

Tot din aceeași familie fac parte și:

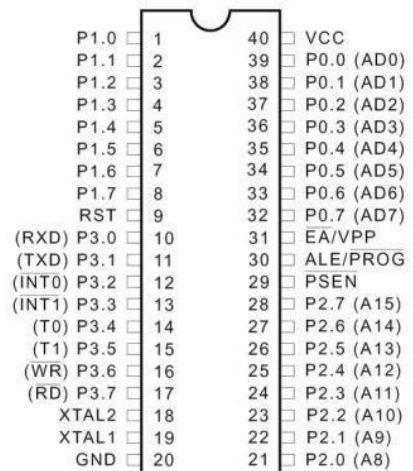
- AT89C51/52 – au caracteristici suplimentare, cum ar fi memorie flash integrată, structură extinsă a întreruperilor și un mod de economisire a energiei;
- STC89C5x – similar cu precedentul, dar are frecvențe de tact mai mari și mai mulți pini de intrare/ieșire. Totuși, dispunerea pinilor și setul de instrucțiuni e diferit față de cel al 8051;

4.2 μ C 8051

Microcontrolerul 8051 este un microcontroler pe 8 biți, bazat pe arhitectura Harvard (memoria de date și memoria programului sunt separate). Aceasta integrează pe un singur cip un procesor (CPU), memorie RAM, memorie ROM, porturi de intrare/ieșire (I/O), temporizatoare și porturi pentru comunicație serială.

Caracteristici:

- Unitatea centrală de procesare de 8 biți este optimizată pentru aplicații de control.
- Procesorul boolean inclus are capabilități extinse pentru prelucrări logice pe bit.



- 32 linii intrare/ ieșire (I/O) bidirecționale cu posibilități de adresare individuală
- 128 octeți RAM intern
- 2 timere/numărătoare pe 16 biți
- Posibilitatea de lucru full-duplex prin interfață serială UART
- 5 surse de întrerupere cu două nivele de priorități
- Generator de tact intern
- 4 Kb de ROM intern
- Posibilitatea de adresare 64 Kb memorie program
- Posibilitatea de adresarea 64 Kb memorie de date
- Set 111 instrucțiuni (dintre care 64 de un singur ciclu mașină), incluzând și instrucțiuni de înmulțire și împărțire.

Există mai mulți producători care oferă diferite funcționalități ale acestui microcontroler:

Microcontroler	Producător	Frecvență maximă	Memorie Flash	RAM	Porturi I/O	Canale ADC integrate	Preț
AT89C51	Atmel	24 MHz	4 KB	128 B	32	Nu	45.59 RON
STC89C5RC	STC	35 MHz	8 KB	512 KB	32	Da	40.32 RON
P89V51RD2	NXP	40 MHz	64 KB	1 KB	32	Nu	41.15 RON

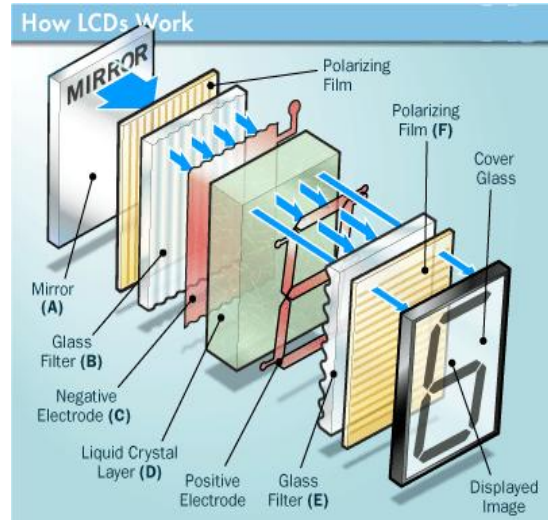
Pentru implementarea proiectului, am să aleg AT89C51, deoarece este cel mai ușor disponibil pe piață și caracteristicile sale sunt suficiente pentru realizarea unui termostat, fiind, de asemenea, facil de programat, datorită arhitecturii cunoscute.

4.3 Principiu de funcționare LCD display

LCD display (Liquid Crysta Display) este un ecran care afișează litere și cifre, fiind construit dintr-o matrice de celule care utilizează cristale lichide și care devin opace sau își schimbă culoarea sub influența unui curent sau câmp electric. Din punct de vedere fizic, fenomenul se explică prin proprietatea cristalelor lichide de a curge ca un lichid, sau de a fi orientate pe o anumită direcție, ca într-un cristal. Astfel, ele pot influența direcția de polarizare a luminii atunci când sunt puse sub o anumită tensiune electrică.

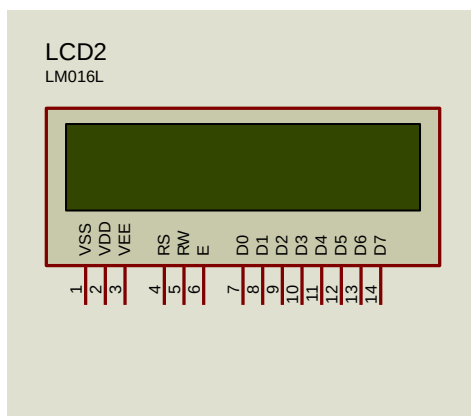
Afișajul LM016L este un modul LCD cu 16 caractere pe 2 linii, echipat cu controlerul HD44780 integrat. Vine la un preț accesibil, de 14.78 RONI. Acesta are următoarele straturi interne:

- **Iluminare de fundal (backlight) :** majoritatea modulelor LM016L include o iluminare de fundal, de obicei realizată cu LED-uri pentru a asigura vizibilitatea în condiții de lumina scăzută.
- **Filtru polarizator frontal :** polarizează lumina vertical.
- **Strat de cristale lichide :** moleculele de cristal lichid își modifică orientarea sub influența tensiunii aplicate, determinând modul în care aceasta este transmisă sau blocată în fiecare segment al afișajului
- **Strat reflectorizant:** are rolul de a reflecta lumina ambientală care trece prin afișaj, permițând utilizatorului să vadă informațiile afișate fără a necesita o sursă de iluminare de fundal.
- **Straturi de electrozi pozitivi și negativi:** plasați în jurul stratului cu cristale lichide, acești electrozi transparenți din oxid de indiu-staniu (ITO) sunt utilizați pentru a controla orientarea moleculelor de cristale lichide . Aceștia formează un camp electric atunci când se aplică o tensiune între electrodul pozitiv și cel negativ. Orientarea moleculelor de cristale lichide determină modul în care lumina este transmisă sau blocată, ceea ce permite afișarea caracterelor pe ecran.



Controlerul **HD44780** gestionează aplicarea tensiunilor necesare între electrozi, interpretând comenzile și datele primite de la microcontrolerul gazdă. Prin aplicarea unor tensiuni specifice la electrozii pozitivi și negativi, controlerul poate activa sau dezactiva segmentele corespunzătoare fiecărui caracter, permițând astfel afișarea informațiilor dorite pe ecran.

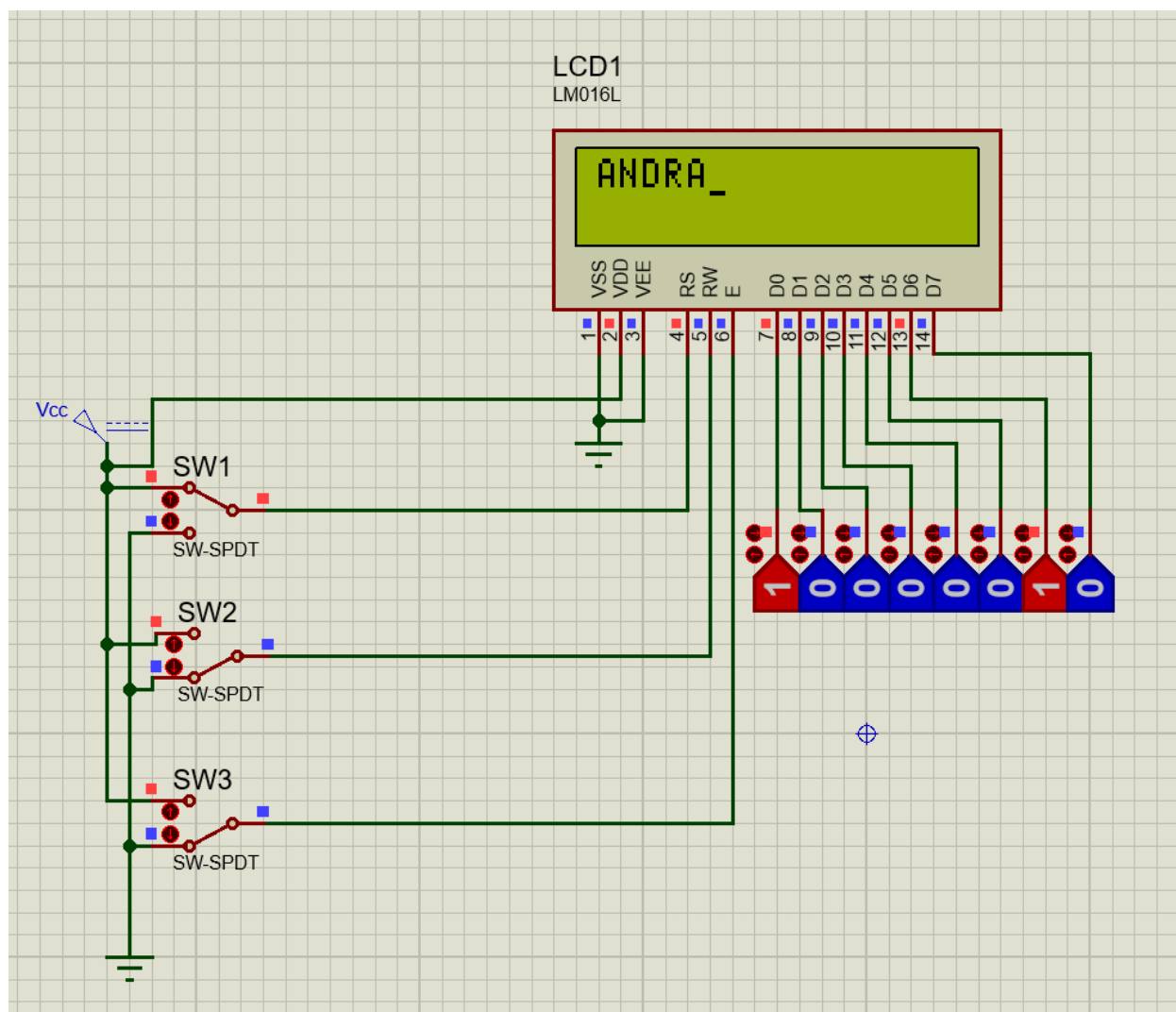
Afișajul LCD este comandat electronic printr-un decodificator de caractere numerice și alfabetice.



Descriere pini LM016L:

- D0 – D7 – Magistrala de date
- R/W – 0 = Scriere în modulul LCD
- 1 = Citire din modulul LCD
- RS – 0 = Intrare instrucțiune and 1 = Intrare date
- VEE – Ajustare contrast
- VSS – Alimentare (GND)
- VCC - Alimentare (+5V)

Următoarea **schema electrică** este utilizată pentru a scrie un mesaj pe LCD:



Switch-urile sunt conectate la RS, RW și E, în timp ce pini D0 – D7 ai LCD-ului sunt conectați la niște stări logice care vor lua valori din următorul tabel pentru a scrie caractere pe ecran:

	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
xxxx0000	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
xxxx0001	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/	:
xxxx0010	;	<	=	>	?@	[	\	]	^	_	`	{		}	~	
xxxx0011																
xxxx0100																
xxxx0101																
xxxx0110																
xxxx0111																
xxxx1000																
xxxx1001																
xxxx1010																
xxxx1011																
xxxx1100																
xxxx1101																
xxxx1110																
xxxx1111																

Switch-ul RS este conectat la pinul **RS** al LCD-ului. Dacă acesta are valoarea **0**, modulul LCD va primi o **instrucțiune**, iar dacă are valoarea **1**, îi vor fi **transmise** date.

Switch-ul E (Enable) este conectat la pinul E. Când acesta este setat la **1**, începe **schimbul de date** între LCD și microcontroler.

Pinul **RW** determină direcția transferului de date:

- 0 – Scriere în LCD
- 1 – Citire din LCD

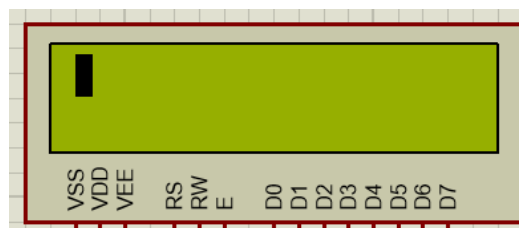
În acest caz, RW va fi conectat la masă (GND), ceea ce înseamnă că modulul LCD va primi date.

Activarea cursorului intermitent:

Pentru a porni cursorul și a-l face să clipească intermitent, inițial **RS**, **RW** și **E** sunt conectate la **GND**.

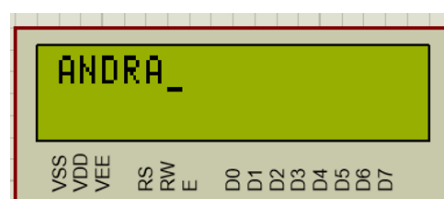
Apoi, stările logice vor lua valoarea **14**, iar switch-ul **E** va fi activat pentru a transmite datele. Următorul pas este setarea valorii **15** pe stările logice, ceea ce va activa cursorul intermitent, conform tabelului de mai jos:

Display on Cursor on	0xE	14
Display on Cursor off	0xC	12
Display on Cursor blinking	0xF	15



Scrierea mesajului pe LCD:

În timpul afișării unui mesaj, switch-ul **RS** rămâne activat, iar stările logice sunt modificate conform tabelului de conversie pentru a reda caracterele dorite. După fiecare caracter încărcat, switch-ul **E** este activat pentru a confirma transferul către LCD.



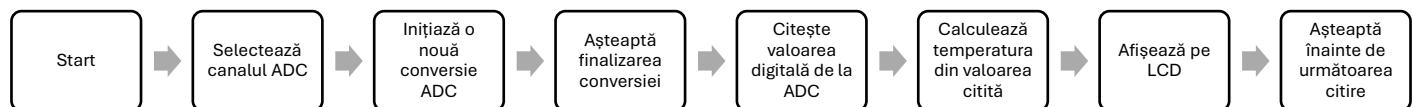
5. Schema electrică a termostatlui și programarea acestuia

Scopul programului care urmează a fi prezentat este implementarea funcționalităților precum: comunicarea cu LCD-ul, citirea de la ADC și generarea semnalului de ceas, în vederea afișării pe LCD a temperaturii prezentate la nivelul senzorului de temperatură.

Fluxul semnalului în schema electrică:

Senzorul de Temperatură convertește temperatura ambientală într-o tensiune analogică proporțională. Urmează **circuitul de amplificare**, folosit pentru a amplifica semnalul de la blocul precedent, cu o amplificare egală cu 20. **Convertorul Analog-Digital** primește semnalul analogic și, sub controlul microcontrolerului, îl transformă într-o valoare digitală pe 8 biți. ADC-ul este multiplexat pe 8 canale, selectate prin pinii de adresă. **Microcontrolerul** generează semnalele de control (START, OE, ADDRESS) pentru ADC, citește și procesează datele digitale de la ieșirea acestuia (prin Portul 1) și generează semnalele și datele necesare pentru a controla afișajul LCD. **Oscilatorul de Sistem** generează semnalul de ceas la o frecvență stabilă, esențial pentru sincronizarea operațiunilor microcontrolerului. Cristalul (X1) definește frecvența, iar condensatoarele (C3, C4) sunt condensatoare de sarcină pentru funcționarea corectă a oscilatorului. **Afișajul** primește datele și comenzile de la microcontroler și afișează valoarea numerică a temperaturii, iar **condensatoarele de decuplare de 1nF**, plasate lângă VCC, asigură stabilitatea tensiunii de alimentare a circuitelor integrate prin filtrarea zgomotului de comutație și furnizarea locală de curent instantaneu.

Fluxul programului:



Explicația elementelor cheie din cod:

- **Selecție canal ADC:** canalul 1 este selectat, prin combinația binară 001.

```
chc = 0;  
chb = 0;  
cha = 1;
```

- **Inițiere conversie ADC:** generează impulsuri simultane pe pinii ALE (care comandă ADC-ului să memorize adresa canalului selectat anterior) și START(care inițiază procesul de conversie analog-digitală pentru canalul memorat) ai ADC-ului.

```
ale = 1;
start = 1;
delay(1);
ale = 0;
start = 0;
```

- **Așteaptă finalizarea conversiei:** la inițierea conversiei, EOC e în stare low, iar când conversia este completă, acesta este high.

```
while(eoc == 1);
while(eoc == 0);
```

- **Citește valoarea digitală de la ADC:** după ce EOC a semnalat finalizarea conversiei, pinul OE este setat pe high. Acesta activează bufferele de ieșire pe 8 biți ale ADC-ului, punând rezultatul conversiei pe pinii săi de date (D0-D7). Valoarea prezentă pe acești pini este apoi citită de microcontroler prin Portul. Imediat după citire, OE este setat înapoi pe low pentru a dezactiva ieșirile ADC-ului, prevenind potențialele conflicte pe magistrala de date.

```
oe = 1;
adc_value = P1;
oe = 0;
```

- **Calculează Temperatura din valoarea citită:** prima parte convertește valoarea digitală (între 0 și 255) într-o tensiune, presupunând o referință de 5V pentru ADC și împărțind produsul dintre valoarea citită și referință la valoarea maximă a adc-ului, care va fi înmulțită cu 20, amplificarea de la circuitul de amplificare. Apoi, această valoare e împărțită la rândul său la 0.02 pentru a convertii valoarea din volți/°C în mv/°C.

```
temperature = ((adc_value * 5.0) / (255.0 * 20)) ;
temperature = (temperature / 0.02); // Convert volts to °C (20mV/°C)
```

- **Afișează Temperatura pe LCD:** se trimite o comandă LCD (0x85) pentru a poziționa cursorul la o anumită locație pe afișaj (a 6-a poziție de pe prima linie) și se scrie mesajul 'Temp:'. Apoi, se apelează funcția print() care este responsabilă de formatarea valorii float din variabila temperature într-un șir de caractere (cifre, punct zecimal, simbol grad, 'C') și trimiterea acestor caractere către LCD pentru afișare, după care se introduce un delay pentru a controla frecvența de lucru cu care sistemul citește și actualizează temperatura afișată.

```
lcd_command(0x85); // Set cursor after "Temp: "
print(temperature); // Display temperature

delay(500); // Delay between readings
```

5.1 Programarea în C

```
#include <reg51.h>
#include <string.h>
```

```

// LCD Control Pins
sbit RS = P2^5;
sbit RW = P2^6;
sbit EN = P2^7;

// ADC Control Pins
sbit ale  = P2^3;
sbit oe   = P2^4;
sbit start = P2^1;
sbit eoc  = P2^0;
sbit clk  = P2^2;

// ADC Channel Selection (assume CH0 is used)
sbit chc = P0^7;
sbit chb = P0^6;
sbit cha = P0^5;

void delay(int t);
void lcd_init(void);
void lcd_command(char c);
void lcd_data(char d);
void str(char a[]);
void print(float p);

float temperature;
unsigned char adc_value;

// Timer0 Interrupt: Generate clock for ADC at 11.0592 MHz
void timer0() interrupt 1 {
    clk = ~clk;
}

void main() {
    lcd_init();
    lcd_command(0x01); // Clear display
    str("Temp: ");
    lcd_command(0x8B); // Set cursor to position for temperature

    // ADC setup
    eoc = 1;
    ale = 0;
    oe = 0;
    start = 0;

    // Timer 0 setup for ADC clock
    TMOD = 0x02; // Timer0 in mode 2 (auto-reload)
    TH0 = 0x00; // Load high byte

```

```

TL0 = 0x00;    // Load low byte
IE = 0x82;    // Enable Timer0 interrupt
TR0 = 1;      // Start Timer0

while(1) {
    // Select Channel 1
    chc = 0;
    chb = 0;
    cha = 1;

    // Start ADC conversion
    ale = 1;
    start = 1;
    delay(1);
    ale = 0;
    start = 0;

    while(eoc == 1); // Wait until EOC goes low
    while(eoc == 0); // Wait until EOC goes high again

    oe = 1;
    adc_value = P1; // Read ADC value (0-255)
    oe = 0;

    // Convert ADC value to temperature in Celsius
    temperature = ((adc_value * 5.0) / (255.0 * 20)) ;
    temperature = (temperature / 0.02); // Convert volts to °C (20mV/°C)

    // Make temperature in reasonable bounds
    if(temperature < 0) temperature = 0;
    if(temperature > 30) temperature = 30;

    lcd_command(0x85); // Set cursor after "Temp: "
    print(temperature); // Display temperature

                                delay(500); // Delay between readings
}
}

// Display a string on the LCD
void str(char a[]) {
    int j;
    for(j = 0; a[j] != '\0'; j++) {
        lcd_data(a[j]);
    }
}

```

```

// LCD Initialization
void lcd_init(void) {
    lcd_command(0x38); // 8-bit, 2-line mode
    lcd_command(0x01); // Clear display
    lcd_command(0x0C); // Display ON, cursor OFF
    lcd_command(0x06); // Entry mode
    lcd_command(0x80); // Force cursor to beginning of first line
}

// Send a command to the LCD
void lcd_command(char c) {
    P3 = c;
    RS = 0;
    RW = 0;
    EN = 1;
    delay(5);
    EN = 0;
    delay(5);
}

// Send a character to the LCD
void lcd_data(char d) {
    P3 = d;
    RS = 1;
    RW = 0;
    EN = 1;
    delay(5);
    EN = 0;
    delay(5);
}

// Simple delay function
void delay(int t) {
    int j, k;
    for(j = 0; j < t; j++) {
        for(k = 0; k < 1275; k++);
    }
}

// Display a float number on the LCD with 1 decimal place
void print(float temp) {
    unsigned char t1, t2, d;

    // Extract integer and decimal parts
    int temp_int = (int)temp;
    int temp_dec = (int)((temp - temp_int) * 10);

```

```

    // Ensure decimal is positive (in case of float errors)
    if (temp_dec < 0) temp_dec = -temp_dec;
    if (temp_dec > 9) temp_dec = 9;

    // Tens digit (only display if temperature is >= 10.0°C)
    if(temp_int >= 10) {
        t1 = (temp_int / 10) + '0';
        lcd_data(t1);
    } else {
        lcd_data(' '); // Blank space if <10°C
    }

    // Units digit
    t2 = (temp_int % 10) + '0';
    lcd_data(t2);

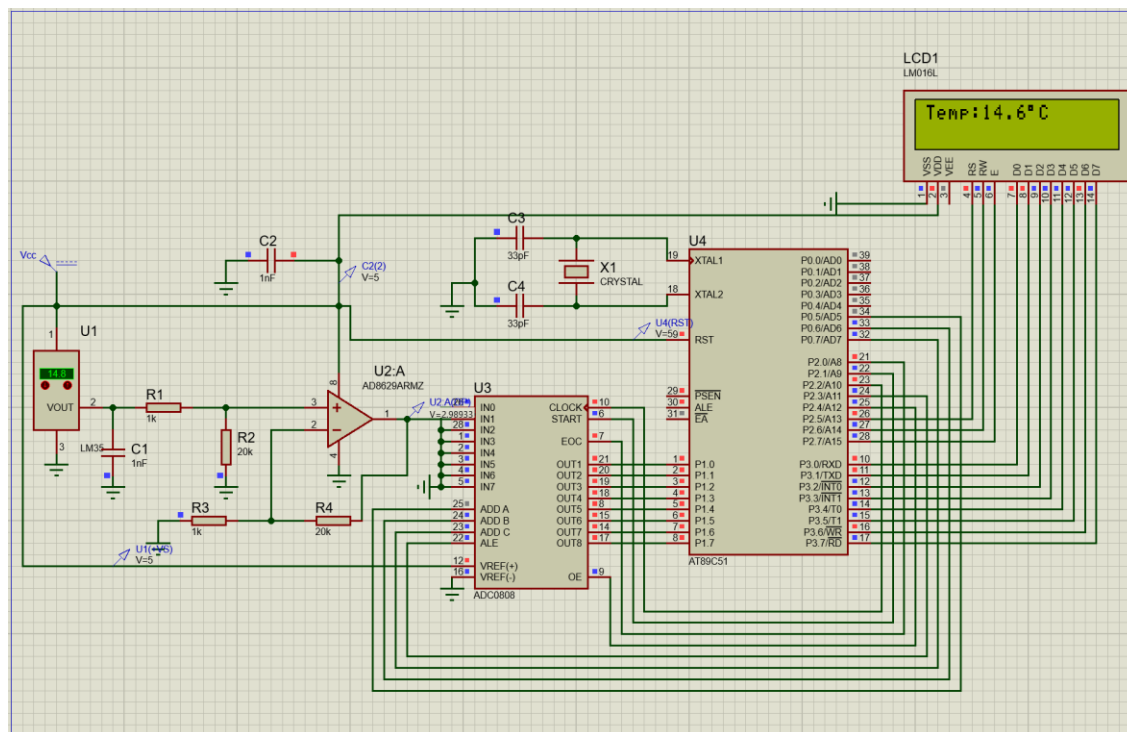
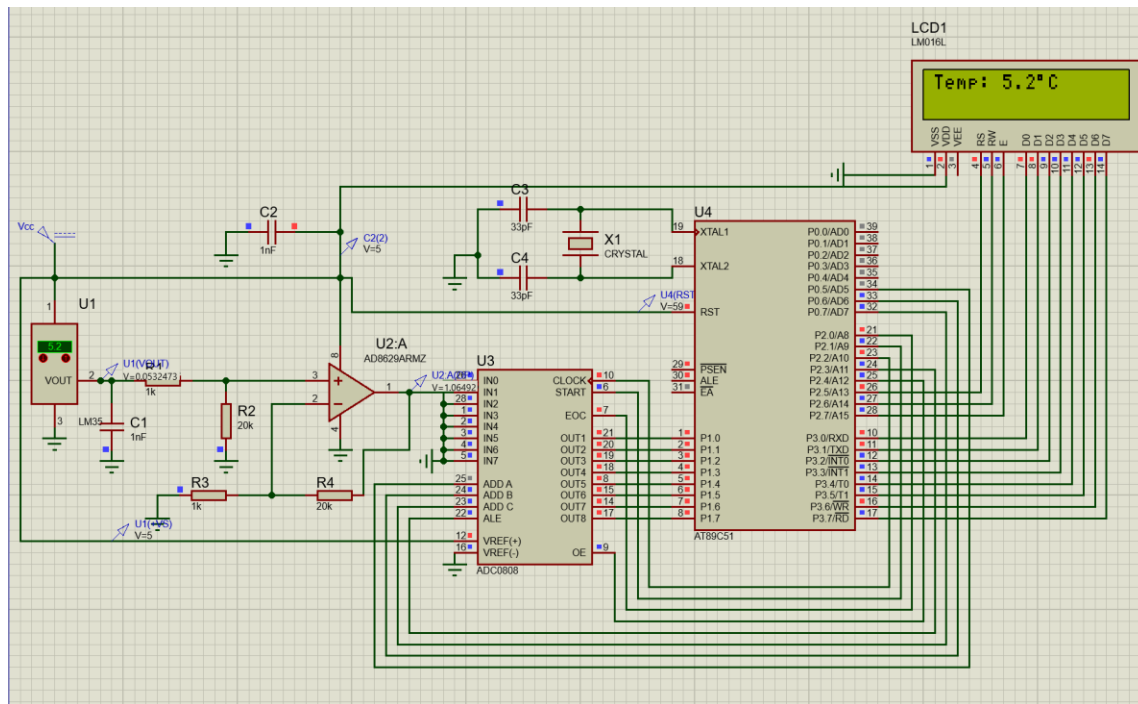
    // Decimal point
    lcd_data('.');

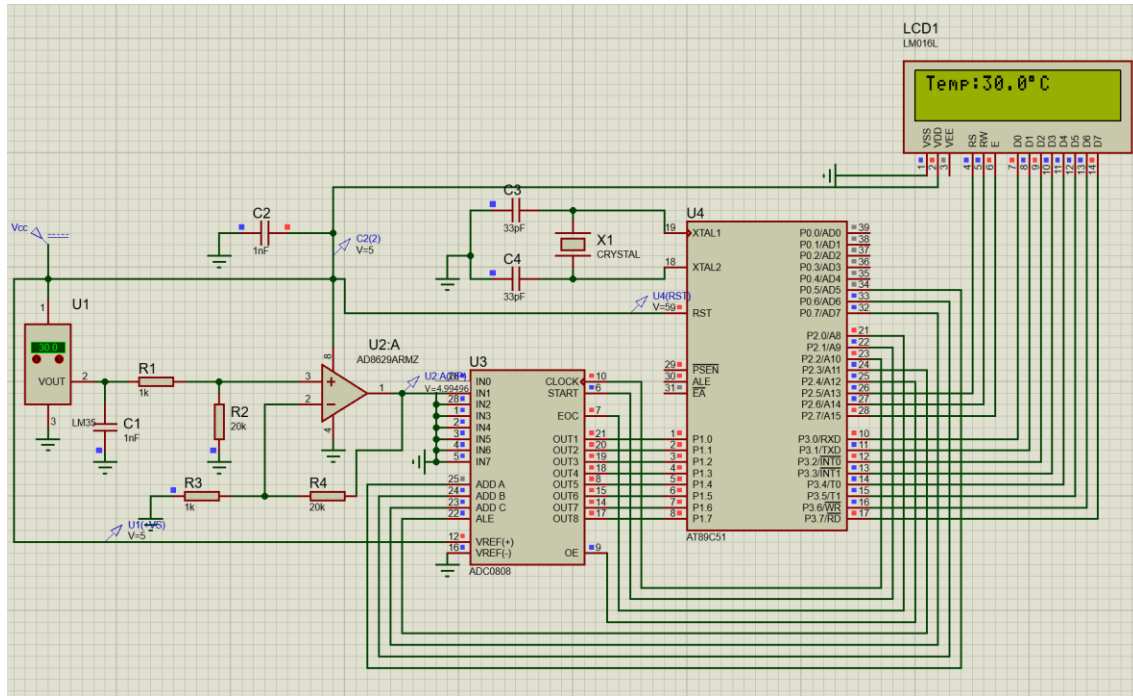
    // Decimal digit
    d = temp_dec + '0';
    lcd_data(d);

    lcd_data(0xDF); // degree symbol
    lcd_data('C');
}

```

Rezultate experimentale:





5.2 Cod în limbaj de asamblare

ORG 0000H

; LCD Control Pins

LCD_RS EQU P2.5

LCD_RW EQU P2.6

LCD_EN EQU P2.7

; ADC Control Pins

ADC_ALE EQU P2.3

ADC_OE EQU P2.4

ADC_START EQU P2.1

ADC_EOC EQU P2.0

ADC_CLK_PIN EQU P2.2

; LCD Initialization

MOV A, #38H ; Function set: 2 lines, 5x7 font

ACALL LCD_CMD_SEND

ACALL DELAY_LONG

MOV A, #0EH ; Display ON, cursor ON

ACALL LCD_CMD_SEND

ACALL DELAY_LONG

MOV A, #01H ; Clear LCD Display

ACALL LCD_CMD_SEND

ACALL DELAY_LONG

MOV A, #06H ; Entry mode: Increment cursor

ACALL LCD_CMD_SEND

ACALL DELAY_LONG

MOV A, #80H ; Set DDRAM address to 0 (first line)

ACALL LCD_CMD_SEND

ACALL DELAY_LONG

; Display Static Text "TEMP: "

MOV A, #'T'

ACALL LCD_DATA_SEND

ACALL DELAY_LONG

MOV A, #'E'

ACALL LCD_DATA_SEND

ACALL DELAY_LONG

MOV A, #'M'

ACALL LCD_DATA_SEND

ACALL DELAY_LONG

```
MOV A, #'P'
ACALL LCD_DATA_SEND
ACALL DELAY_LONG
```

```
MOV A, #':'
ACALL LCD_DATA_SEND
ACALL DELAY_LONG
```

```
MOV A, #' '
ACALL LCD_DATA_SEND
ACALL DELAY_LONG
```

```
; Timer0 Initialization
MOV TMOD, #02H    ; Timer0 in mode 2 (auto-reload)
MOV TH0, #0FFH    ; Set Timer0 high byte for reload
MOV TL0, #0FFH    ; Initial timer low byte
MOV IE, #82H      ; Enable Timer0 interrupt and global interrupt
SETB TR0          ; Start Timer0
```

```
; Main Application Loop
```

```
MAIN_LOOP:
```

```
    ; Select ADC Channel
```

```
    CLR P2.2      ; C=0
```

```
    CLR P2.1      ; B=0
```

```
    SETB P2.0     ; A=1
```

```
    ; Start ADC conversion
```

```
    CLR ADC_ALE
```

```
    CLR ADC_START
```

```
    SETB ADC_ALE
```

```
    SETB ADC_START
```

```
    ACALL DELAY_SHORT
```

CLR ADC_ALE

CLR ADC_START

WAIT_EOC_LOW:

JB ADC_EOC, WAIT_EOC_LOW

WAIT_EOC_HIGH:

JNB ADC_EOC, WAIT_EOC_HIGH

; Read ADC Output

SETB ADC_OE

ACALL DELAY_SHORT

MOV A, P1 ; Read ADC 8-bit output from Port1

MOV R5, A ; Save ADC value to R5

CLR ADC_OE ; Clear OE after read

; Calculate approximate temperature

MOV A, R5

MOV B, #5

MUL AB ; A = ADC value * 5

MOV B, #255

DIV AB ; A = (ADC*5)/255

MOV R4, A ; Save temperature to R4

; Move LCD cursor after "TEMP: "

MOV A, #85H

ACALL LCD_CMD_SEND

ACALL DELAY_LONG

; Display tens and unit digits

MOV A, R4

ACALL DISPLAY_TU

```

MOV A, #0DFH    ; Degree symbol (custom char)
ACALL LCD_DATA_SEND
ACALL DELAY_LONG
MOV A, #'C'
ACALL LCD_DATA_SEND
ACALL DELAY_LONG

SJMP MAIN_LOOP ; Repeat

; Subroutine: Display Tens and Units of Binary Number
DISPLAY_TU:
    MOV B, #10
    DIV AB      ; A = Tens digit, B = Units digit
    ADD A, #30H ; Convert tens to ASCII
    ACALL LCD_DATA_SEND
    ACALL DELAY_SHORT

    MOV A, B
    ADD A, #30H ; Convert units to ASCII
    ACALL LCD_DATA_SEND
    ACALL DELAY_SHORT
    RET

; Subroutine: Send Command to LCD
LCD_CMD_SEND:
    MOV P3, A    ; Send command to Port 3
    CLR LCD_RS   ; RS=0 for command
    CLR LCD_RW   ; RW=0 for write
    SETB LCD_EN  ; EN=1 to latch command
    ACALL DELAY_SHORT
    CLR LCD_EN   ; EN=0 complete latch
    RET

```

; Subroutine: Send Data to LCD

LCD_DATA_SEND:

```
MOV P3, A      ; Send data to Port 3
SETB LCD_RS    ; RS=1 for data
CLR LCD_RW     ; RW=0 for write
SETB LCD_EN    ; EN=1 to latch data
ACALL DELAY_SHORT
CLR LCD_EN     ; EN=0 complete latch
RET
```

DELAY_SHORT:

```
MOV R0, #50
```

LOOP1:

```
MOV R1, #255
```

LOOP2:

```
DJNZ R1, LOOP2
```

```
DJNZ R0, LOOP1
```

```
RET
```

DELAY_LONG:

```
MOV R2, #100
```

DELAY_LOOP1:

```
MOV R3, #255
```

DELAY_LOOP2:

```
DJNZ R3, DELAY_LOOP2
```

```
DJNZ R2, DELAY_LOOP1
```

```
RET
```

; Timer0 Interrupt Vector

```
ORG 000BH      ; Timer0 interrupt vector address
```

TIMER0_ISR:

CPL ADC_CLK_PIN ; Toggle ADC Clock (generate square wave)

RETI ; Return from interrupt

END

